

1. Architecture logique du programme

Ton firmware fait 3 choses :

boucle principale :

- clignote GPIO2 :
 - 2 blinks rapides
 - 2 blinks longs
 - 2 secondes ON/OFF
 - affiche logs UART
-

2. Rôle de chaque fichier

2.1 esp32_regs.h (LE CŒUR BAS NIVEAU)

rôle

C'est ton **hardware abstraction manuel**

👉 tu y définis les adresses mémoire des registres ESP32

explication

```
#define GPIO_ENABLE_REG (*(volatile uint32_t *)0x3FF44020)
```

- `0x3FF44020` → adresse physique du registre GPIO enable
- `(volatile uint32_t *)` :
 - volatile = la valeur peut changer matériellement
 - empêche le compilateur d'optimiser
- `*` :
 - pointe vers une adresse mémoire

- tu accèdes directement à la mémoire

👉 donc :

écrire dans cette variable = écrire dans le hardware ESP32

exemples importants

GPIO_OUT_W1TS_REG

```
#define GPIO_OUT_W1TS_REG (*(volatile uint32_t *)0x3FF44008)
```

👉 W1TS = Write 1 To Set

- ✓ écrire 1 → met un pin à HIGH
 - ✓ pas besoin de read-modify-write
-

UART registers

```
#define UART_FIFO_REG (*(volatile uint32_t*)(UART0_BASE + 0x00))
```

👉 FIFO = buffer d'envoi UART

2.2 gpio.c (contrôle matériel GPIO)

rôle

Piloter GPIO2 directement via registres

gpio_init_output()

objectif

configurer un pin comme sortie GPIO

ligne importante

```
volatile uint32_t *iomux = (volatile uint32_t *)0x3FF49040;
```

👉 IO MUX = sélection de fonction du pin

```
*iomux = (2 << 12);
```

explication

- bits 12–14 = fonction du pin
- 2 = GPIO function

👉 donc :

force le pin à devenir un GPIO

GPIO enable

```
GPIO_ENABLE_W1TS = (1U << pin);
```

👉 active le pin en sortie

gpio_set()

```
GPIO_OUT_W1TS_REG = (1U << pin);
```

👉 met pin HIGH

gpio_clear()

```
GPIO_OUT_W1TC_REG = (1U << pin);
```

👉 met pin LOW



2.3 uart.c (communication série)



rôle

envoyer texte vers terminal via UART0



uart_putc()

```
while (UART_STATUS_REG & 0x00400000);
```



explication

- vérifie FIFO plein
 - attend que buffer ait de la place
-

```
UART_FIFO_REG = c;
```



envoie un caractère



uart_puts()

```
while (*s)
    uart_putc(*s++);
```



explication

- boucle sur string
 - envoie caractère par caractère
-



2.4 main.c (LOGIQUE APPLICATION)



rôle

✎ orchestrer GPIO + UART + timing

delay_ms()

```
vTaskDelay(pdMS_TO_TICKS(ms));
```

important

- ESP-IDF utilise FreeRTOS
- **vTaskDelay** = sleep sans bloquer CPU complètement

blink_quick()

```
gpio_set(2);  
delay_ms(200);  
gpio_clear(2);  
delay_ms(200);
```

 un blink rapide

blink_long()

 même logique mais 800 ms

hold_steady()

 LED ON 2s puis OFF 2s

app_main()

rôle

point d'entrée ESP-IDF (comme main() mais RTOS)

UART init

```
uart_init();  
gpio_init_output(2);
```

👉 initialise hardware

messages UART

```
uart_puts("=== ESP32 QEMU Bare-Metal ===");
```

👉 debug console

boucle infinie

```
while (1)
```

👉 firmware embarqué = jamais fini

cycle complet

```
2 quick  
2 long  
hold  
repeat
```

2.5 CMakeLists.txt (TRÈS IMPORTANT)

rôle

👉 système de build



Root CMakeLists.txt

```
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(esp32_qemu_lab)
```



explication

- charge ESP-IDF build system
- définit le projet



ESP-IDF = framework basé sur CMake



main/CMakeLists.txt

```
idf_component_register(SRCS "main.c" "gpio.c" "uart.c"
                        INCLUDE_DIRS ".")
```



explication

- SRCS → fichiers compilés
- INCLUDE_DIRS → headers



ça crée un “component ESP-IDF”



3. Comment tout fonctionne ensemble



pipeline complet

1. Build

idf.py build



CMake :

- compile code
- link ESP-IDF runtime

- génère firmware
-

2. Flash image (QEMU ou ESP32)

esptool.py merge_bin

👉 assemble :

- bootloader
 - partition table
 - app
-

3. Execution

- QEMU ou hardware ESP32
 - bootloader démarre
 - appelle `app_main()`
-

4. runtime

Ton code tourne :

UART init

GPIO init

loop:

 blink

 print UART

OK, on va faire ça proprement et clairement :

1. 🧱 À quoi sert CMake (simple + concret)
 2. 📄 Commentaires **ligne par ligne** de TES fichiers
 3. 🧠 rôle global de chaque code (résumé clair)
-

🧱 5. CMAKE → ça sert à quoi exactement ?

idée simple :

👉 CMake = **chef d'orchestre de compilation**

Il ne compile pas ton code.

Il dit au compilateur :

“prends ces fichiers, compile-les comme ça, link-les avec ESP-IDF, et génère un firmware”

sans CMake :

Tu devrais faire manuellement :

xtensa-esp-elf-gcc main.c gpio.c uart.c ...

ld ...

objcopy ...

❌ impossible à gérer dans un projet réel

avec CMake (ESP-IDF)

Tu fais juste :

idf.py build

Et CMake fait automatiquement :

- 1. trouve ESP-IDF**
 - 2. configure toolchain Xtensa**
 - 3. compile fichiers .c**
 - 4. link avec FreeRTOS + bootloader**
 - 5. génère .bin**
-

résumé ultra simple :

CMake	rôle
organise	fichiers source
configure	compilation
automatise	build firmware
connecte	ESP-IDF + GCC